

Phase 1: Installation & Setup

1. Install Java & Verify

Code:

```
sudo apt update
```

```
sudo apt install openjdk-21-jdk -y
```

```
echo 'export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64' >> ~/.bashrc  
source ~/.bashrc
```

```
java -version
```

```
cd /tmp
```

```
wget https://dlcdn.apache.org/spark/spark-4.1.1/spark-4.1.1-bin-hadoop3.tgz
```

```
tar xzf spark-4.1.1-bin-hadoop3.tgz
```

```
sudo mv spark-4.1.1-bin-hadoop3 /opt/spark
```

```
echo 'export SPARK_HOME=/opt/spark' >> ~/.bashrc  
echo 'export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin' >> ~/.bashrc  
echo 'export PYSARK_PYTHON=/usr/bin/python3' >> ~/.bashrc
```

```
source ~/.bashrc
```

```
echo $SPARK_HOME
```

Phase 2: Your First Spark Exercise

```
pyspark
```

```
data = spark.sparkContext.parallelize([  
    "Hello world Hello Spark",  
    "Spark is great for big data",  
    "Hello Python and Spark"  
])
```

```
df = data.toDF(schema=["line"])
```

```
from pyspark.sql.functions import explode, split, col, desc
```

```
words_df = df.select(explode(split(col("line"), " ")).alias("word"))
```

```
word_counts = words_df.groupBy("word").count()

sorted_word_counts = word_counts.orderBy(desc("count"))

sorted_word_counts.show()
```

Output:

```
pratham@LAPTOP-PL4G895R:~$ sudo apt update
[sudo: authenticate] Password:
Hit:1 https://download.docker.com/linux/ubuntu noble InRelease
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
pratham@LAPTOP-PL4G895R:~$ sudo apt install openjdk-21-jdk -y
openjdk-21-jdk is already the newest version (21.0.11+10-1~26.04.2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2
```

```
pratham@LAPTOP-PL4G895R:~$ echo 'export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64' >> ~/.bashrc
pratham@LAPTOP-PL4G895R:~$ source ~/.bashrc
pratham@LAPTOP-PL4G895R:~$ java -version
openjdk version "21.0.11" 2026-04-21
OpenJDK Runtime Environment (build 21.0.11+10-1-26.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.11+10-1-26.04.2-Ubuntu, mixed mode, sharing)
```

```
pratham@LAPTOP-PL4G895R:/tmp$ cd /tmp
wget https://downloads.apache.org/spark/spark-4.1.2/spark-4.1.2-bin-hadoop3.tgz
--2026-07-28 11:01:46-- https://downloads.apache.org/spark/spark-4.1.2/spark-4.1.2-bin-hadoop3.tgz
Resolving downloads.apache.org (downloads.apache.org)... 88.99.208.237, 95.216.224.44, 2a01:4f8:10a:39da::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|88.99.208.237|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 572818974 (546M) [application/x-gzip]
Saving to: 'spark-4.1.2-bin-hadoop3.tgz'

spark-4.1.2-bin-hadoop3.tgz      100%[=====] 546.28M  8.13MB/s   in 75s
2026-07-28 11:03:02 (7.31 MB/s) - 'spark-4.1.2-bin-hadoop3.tgz' saved [572818974/572818974]
```

```
pratham@LAPTOP-PL4G895R:/tmp$ cd /tmp
tar xzf spark-4.1.1-bin-hadoop3.tgz
tar (child): spark-4.1.1-bin-hadoop3.tgz: Cannot open: No such file or directory
tar (child): Error is not recoverable: exiting now
tar: Child returned status 2
tar: Error is not recoverable: exiting now
pratham@LAPTOP-PL4G895R:/tmp$ sudo mv spark-4.1.1-bin-hadoop3 /opt/spark
mv: cannot stat 'spark-4.1.1-bin-hadoop3': No such file or directory
pratham@LAPTOP-PL4G895R:/tmp$ ls /opt
containerd  spark
pratham@LAPTOP-PL4G895R:/tmp$ echo 'export SPARK_HOME=/opt/spark' >> ~/.bashrc
pratham@LAPTOP-PL4G895R:/tmp$ echo 'export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin' >> ~/.bashrc
pratham@LAPTOP-PL4G895R:/tmp$ echo 'export PYSPARK_PYTHON=/usr/bin/python3' >> ~/.bashrc
pratham@LAPTOP-PL4G895R:/tmp$ source ~/.bashrc
pratham@LAPTOP-PL4G895R:/tmp$ echo $SPARK_HOME
/opt/spark
```

```

pratham@LAPTOP-PL4G895R:/tmp$ spark-shell --version
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/07/28 11:04:14 WARN Utils: Your hostname, LAPTOP-PL4G895R, resolves to a loopback address: 127.0.1.1; using 10.255.255.254 instead (on interface lo)
26/07/28 11:04:14 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Welcome to

  ____      _
 / ___|  ___| | | |
| |___| |___| | | |
|___|_|___|_|_|_|

 version 4.0.1

Using Scala version 2.13.16, OpenJDK 64-Bit Server VM, 21.0.11
Branch HEAD
Compiled by user runner on 2025-09-02T03:10:51Z
Revision 29434ea766b0fc3c3bf6eaadb43a8f931133649e
Url https://github.com/apache/spark
Type --help for more information.
pratham@LAPTOP-PL4G895R:/tmp$ echo $SPARK_HOME
/opt/spark

```

```

pratham@LAPTOP-PL4G895R:/tmp$ pyspark
Python 3.14.4 (main, Jun 18 2026, 14:25:02) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/07/28 11:10:34 WARN Utils: Your hostname, LAPTOP-PL4G895R, resolves to a loopback address: 127.0.1.1; using 10.255.255.254 instead (on interface lo)
26/07/28 11:10:34 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/07/28 11:10:34 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Welcome to

  ____      _
 / ___|  ___| | | |
| |___| |___| | | |
|___|_|___|_|_|_|

 version 4.0.1

Using Python version 3.14.4 (main, Jun 18 2026 14:25:02)
Spark context Web UI available at http://10.255.255.254:4040
Spark context available as 'sc' (master = local[*], app id = local-1785237035681).
SparkSession available as 'spark'.
>>> data = spark.sparkContext.parallelize([
...     ("Hello world Hello Spark",),
...     ("Spark is great for big data",),
...     ("Hello Python and Spark",)
... ])
>>> df = data.toDF(["line"])
>>> df.show()
+-----+
|           line|
+-----+
|Hello world Hello...|
|Spark is great fo...|
|Hello Python and ...|
+-----+

>>> from pyspark.sql.functions import explode, split, col, desc
>>> words_df = df.select(explode(split(col("line"), " ")).alias("word"))
>>> word_counts = words_df.groupBy("word").count()
>>> sorted_word_counts = word_counts.orderBy(desc("count"))

```

```

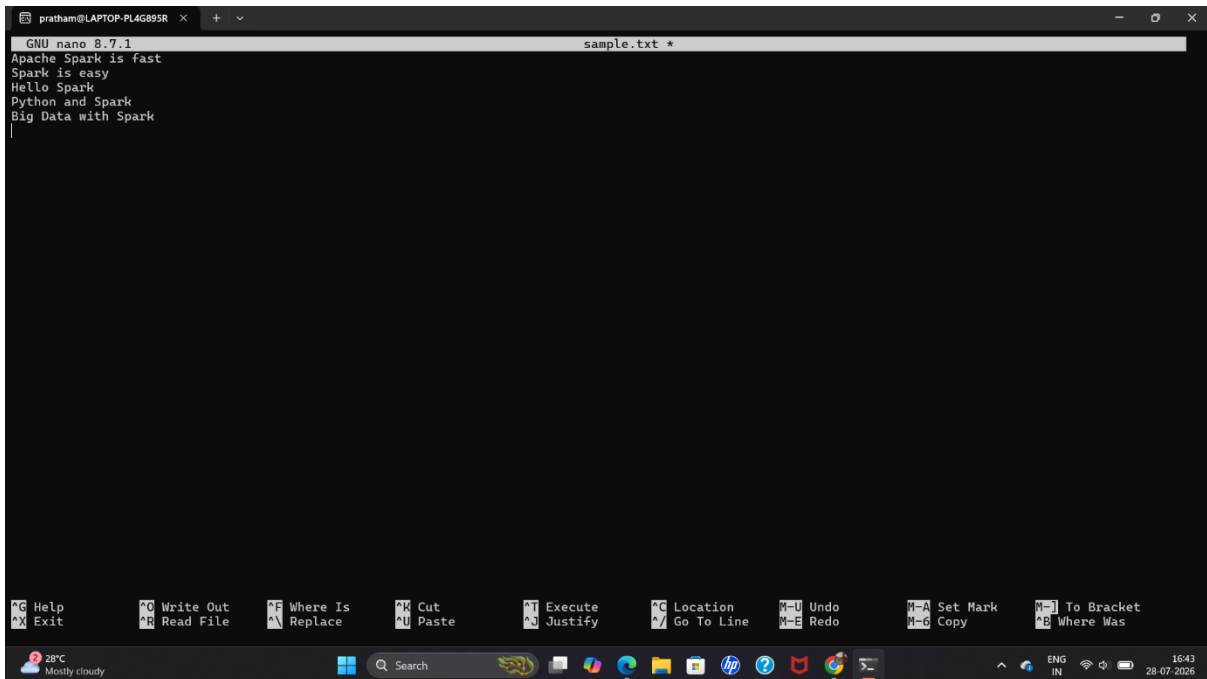
>>> from pyspark.sql.functions import explode, split, col, desc
>>> words_df = df.select(explode(split(col("line"), " ")).alias("word"))
>>> word_counts = words_df.groupBy("word").count()
>>> sorted_word_counts = word_counts.orderBy(desc("count"))
>>> sorted_word_counts.show()
+-----+-----+
| word|count|
+-----+-----+
| Hello|    3|
| Spark|    3|
| world|    1|
|  for|    1|
| great|    1|
|   is|    1|
|  data|    1|
|   big|    1|
|  and|    1|
|Python|    1|
+-----+-----+

>>> exit

```

3. Advanced Task: Word Count with a Text File

```
pratham@LAPTOP-PL4G895R:/tmp$ nano sample.txt
```



```
GNU nano 8.7.1 sample.txt *
Apache Spark is fast
Spark is easy
Hello Spark
Python and Spark
Big Data with Spark
|
```

```
pratham@LAPTOP-PL4G895R:/tmp$ cat sample.txt
Apache Spark is fast
Spark is easy
Hello Spark
Python and Spark
Big Data with Spark
```

```
pratham@LAPTOP-PL4G895R:/tmp$ pyspark
Python 3.14.4 (main, Jun 18 2026, 14:25:02) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/07/28 11:13:27 WARN Utils: Your hostname, LAPTOP-PL4G895R, resolves to a loopback address: 127.0.1.1; using 10.255.255.254 instead (on interface lo)
26/07/28 11:13:27 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/07/28 11:13:28 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Welcome to

  ____      _
 / ___|  __| | | |
| |  | |__| | | |
| |  | |__| | | |
| |  | |__| | | |
|_|  |____|_|_|_|

version 4.0.1

Using Python version 3.14.4 (main, Jun 18 2026 14:25:02)
Spark context Web UI available at http://10.255.255.254:4040
Spark context available as 'sc' (master = local[*], app id = local-1785237209902).
SparkSession available as 'spark'.
>>> df = spark.read.text('sample.txt')
>>> df.show(truncate=False)
+-----+
|value|
+-----+
|Apache Spark is fast|
|Spark is easy|
|Hello Spark|
|Python and Spark|
|Big Data with Spark|
+-----+
```

```
>>> from pyspark.sql.functions import explode, split, col, desc
>>> words_df = df.select(explode(split(col("value"), " ")).alias("word"))
>>> word_counts = words_df.groupBy("word").count()
>>> sorted_word_counts = word_counts.orderBy(desc("count"))
>>> sorted_word_counts.show()
+-----+-----+
| word | count |
+-----+-----+
| Spark |      5 |
| is    |      2 |
| Data  |      1 |
| with  |      1 |
| fast  |      1 |
| Apache |      1 |
| Hello |      1 |
| and   |      1 |
| Big   |      1 |
| Python |      1 |
| easy  |      1 |
|       |      1 |
+-----+-----+

>>> exit
```

Reflection Questions:

1. DataFrames vs. RDDs: Why might a DataFrame be more efficient than an RDD for this word count task? (Hint: Think about Spark's Catalyst Optimizer.)

A **DataFrame** is generally more efficient than an **RDD** because it uses **Spark's Catalyst Optimizer**, which automatically analyzes and optimizes the execution plan. It can optimize operations such as filtering, grouping, and aggregation, reducing execution time and resource usage. DataFrames also use Spark's optimized execution engine, making them faster and more memory-efficient than RDDs for structured data processing.

2. Lazy Evaluation: In the PySpark shell, when does the actual computation for word_counts happen? Is it when groupBy and count are defined, or when show() is called?

Spark follows the concept of **Lazy Evaluation**. Methods such as groupBy() and count() are **transformations**, so they only build an execution plan and do not perform the computation immediately. The actual computation starts only when an **action** such as show() is called. At that point, Spark executes all the pending transformations and produces the final result.

3. Partitions: How could the spark.sparkContext.parallelize() function be used to control the parallelism of the job?

The parallelize() function accepts an optional **second argument** that specifies the number of **partitions**. More partitions allow Spark to process data in parallel across multiple CPU cores, which can improve performance for larger datasets. Choosing an appropriate number of partitions helps balance the workload and use system resources efficiently.